5.5 Organizing your APIs with route groups

One criticism levied against minimal APIs in .NET 6 was that they were necessarily quite verbose, required a lot of duplicated code, and often led to large endpoint handler methods. .NET 7 introduced two new mechanisms to address these critiques:

- *Filters*—Introduced in section 5.4, filters help separate validation checks and cross-cutting functions such as logging from the important logic in your endpoint handler functions.
- *Route groups*—Described in this section, route groups help reduce duplication by applying filters and routing to multiple handlers at the same time.

When designing APIs, it's important to maintain consistency in the routes you use for your endpoints, which often means duplicating part of the route pattern across multiple APIs. As an example, all the endpoints in the `fruit` API described throughout this chapter (such as in listing 5.3) start with the route prefix `/fruit`:

- `MapGet("/fruit", () => {/* */})`
- `MapGet("/fruit/{id}", (string id) => {/* */})`
- `MapPost("/fruit/{id}", (Fruit fruit, string id) => {/* */})`
- `MapPut("/fruit/{id}", (Fruit fruit, string id) => {/* */})`